

Video Summary

This video introduces Dictionaries in Python, a powerful data structure for storing information in key-value pairs. Unlike lists that rely on index numbers, dictionaries map custom keys to specific values for efficient retrieval.

Source video: https://www.youtube.com/watch?v=_vI4dNCS5bY&list=PLsyebzWxl7omDoEYrrf3oXvXxa6MPgek&index=11

1 Defining Dictionaries

Dictionaries are created with curly brackets and store data as key-value pairs. The key is used to find the value later.



Curly brackets

Use braces when writing dictionary literals.



Key-value pairs

Each key points to a value with a colon.



Custom lookup

Keys replace numeric index lookup for named data.

```
>>> data = {1: "Navin", 2: "Kiran", 4: "Harsh"}
>>> data
{1: "Navin", 2: "Kiran", 4: "Harsh"}
>>> data[4]
'Harsh'
```

dictionary definition

2 Accessing Data by Key

Dictionary values are fetched with their keys. A number is not treated as a position unless that exact number exists as a key.

```
key lookup and KeyError

>>> data = {1: "Navin", 2: "Kiran", 4: "Harsh"}
>>> data[1]
'Navin'
>>> data[3]
KeyError: 3
```

Lookup rule

Use a dictionary key, not a list-style position. If the key is missing, direct lookup raises `KeyError`.

3 The `.get()` Method

`.get()` is a safer way to retrieve dictionary data. Missing keys return `None` by default, or a custom fallback message if you provide one.

Expression	Meaning	Result
<code>data.get(1)</code>	Find key 1 safely.	'Navin'
<code>data.get(3)</code>	Missing key with no fallback.	None
<code>data.get(3, "not found")</code>	Missing key with custom fallback.	not found

4

Unique Keys and Values

Dictionary keys behave like set members: each key must be unique. Values behave more like list items: values can repeat and can also be mutable objects.

1x

Keys are unique

A duplicate key overwrites the earlier value.

val

Values may repeat

Different keys can point to the same value.

[]

Values can vary

Values can be strings, lists, tuples, dicts, and more.

```
>>> data = {1: "Navin", 1: "Kiran", 2: "Kiran"}
>>> data
{1: "Kiran", 2: "Kiran"}
# key 1 kept only the latest value
```

unique key example

5

Creating Dictionaries with zip()

zip() pairs items from two collections. Wrap it with dict() to convert those pairs into a dictionary.

```
>>> keys = ["navin", "kiran", "harsh"]
>>> values = ["python", "java", "js"]
>>> data = dict(zip(keys, values))
>>> data
{"navin": "python", "kiran": "java", "harsh": "js"}
```

zip and dict

6 Removing Data

Dictionaries can remove items by key. `pop()` removes the pair and returns the removed value, while `del` removes the pair without returning it.

Tool	What it does	Example result
<code>data.pop("kiran")</code>	Removes the kiran key and returns its value.	'java'
<code>del data["harsh"]</code>	Deletes the harsh key-value pair.	No returned value
<code>data.clear()</code>	Removes all pairs from the dictionary.	<code>{}</code>

```
removing entries

>>> data = {"navin": "python", "kiran": "java", "harsh": "js"}
>>> data.pop("kiran")
'java'
>>> del data["harsh"]
>>> data
{"navin": "python"}
```

7 Nested Dictionaries

Nested dictionaries store complex data by placing dictionaries inside other dictionaries. This creates a hierarchy that can model grouped information.

```
nested structure

>>> prog = {
...     "JS": "Atom", "CS": "VS",
...     "Python": ["PyCharm", "Sublime"],
...     "Java": {"JSE": "NetBeans", "JEE": "Eclipse"}
... }
>>> prog["Java"]["JEE"]
'Eclipse'
```

8

Exercise Code

This exercise mixes list, dictionary, and tuple values inside one dictionary, then fetches nested values by chaining keys and indexes.

```
nested dictionary exercise

data = {
    'harsh': ['Java', 'Python', 'AI'],
    'navin': {'frotend': 'react', 'backend': 'spring'},
    'kiran': ('js', 'flask')
}

print(data['navin']['backend'])
print(type(data['navin']))

print(data['harsh'][1])
print(print(type(data['harsh'])))

print(data['kiran'][0])
print(type(data['kiran']))
```

Practice focus

The outer keys are 'harsh', 'navin', and 'kiran'. The values are different collection types, so access changes based on the nested value type.

9

Expected Exercise Output

The nested lookup reads through the outer dictionary first, then into the selected inner object.

```
spring
<class 'dict'>
Python
<class 'list'>
None
js
<class 'tuple'>
```

Why None appears

`print(print(type(data['harsh'])))` first prints the list type from the inner `print()`. Then the outer `print()` prints `None` because `print()` itself returns `None`.

Expression	Path	Value or type
<code>data['navin']['backend']</code>	Outer key navin, inner key backend.	spring
<code>data['harsh'][1]</code>	Outer key harsh, list index 1.	Python
<code>data['kiran'][0]</code>	Outer key kiran, tuple index 0.	js

Day 12/55 takeaway

Dictionaries connect custom keys to values. Use keys for lookup, `.get()` for safer retrieval, `zip()` to build from pairs, `pop()` or `del` to remove data, and nesting for structured information.